Plano do Desafio — Sistema de Matrículas

Nov 7, 2026 · @João Moura

1. Visão geral

Entregar até **15/11/2026, 23:59** um front-end React de matrículas com back-end Express, 4 casos de uso funcionando, um ZIP e um vídeo de 3 a 5 min. O desafio vale 6 pontos e permite 3 tentativas.

Tema: sistema de atendimento e matrículas, com visão institucional (monta turmas) e visão do aluno (se matricula). Turma = disciplina + professor + alunos + horário (código de 2 dígitos: dia 2–7 e turno 1–3).

Entregável	Formato	Obrigatório
Código-fonte (front + back)	Um único .zip, sem node_modules	Sim
Planejamento / protótipo	PDF exportado deste doc + imagens das telas	Sim
Vídeo de execução e código	MP4, 3 a 5 min, código legível	Sim
Nomes dos integrantes	Texto na postagem	Sim

Critério	Peso	Como vamos atacar
Funcionalidade e completude	40%	4 casos de uso completos, testados um por vez
Complexidade dos requisitos	30%	7 regras de negócio (conflito de horário, vagas, pré-requisito, carga do professor)
Qualidade geral (código, apresentação)	20%	Componentes pequenos, nomes claros, vídeo roteirizado
Uso das tecnologias estudadas	10%	Classes, state, ciclo de vida, Context, fetch, Error Boundary, Express

Atenção: link de GitHub não é aceito, e sem vídeo ou sem arquivos a nota é zero.

2. Tecnologias permitidas

Usar só o que aparece nos exemplos do professor: React com create-react-app e componentes de classe, Bootstrap, fetch e um servidor Node + Express que grava em arquivo JSON. Cada recurso abaixo deve aparecer no código e ser citado no vídeo.

Recurso	Semana	Onde usar no projeto
Componentes de classe e JSX	2–3	Todas as telas (`class X extends Component`)
state e setState	3	Formulários, listas, tela atual
componentDidMount	3	Carregar dados do back-end ao abrir cada tela
Eventos (onClick, onChange, onSubmit, preventDefault)	3–4	Botões Matricular, Cancelar, Salvar
Listas com .map() e key	4	Tabelas de disciplinas, turmas, alunos
Formulários controlados (input, select)	4	Cadastro de disciplina, professor, turma
Renderização condicional	2, 4	Trocar telas por perfil; mensagens de erro
Composição e props.children	4	Componente Card/Layout reutilizável
Error Boundary	4	Envolver a área principal (componentDidCatch)
Context (createContext)	5	Usuário logado, perfil e tela atual
fetch + .then() + JSON.stringify	5	Todas as chamadas à API
Node + Express + cors	5–6	Pasta backend/ com server.js
fs (readFile/writeFile em JSON)	5	Persistir dados em dados.json
create-react-app	6	Estrutura do projeto front
Bootstrap	4 (vídeo) + enunciado	Layout, tabelas, formulários, alertas

Evitar (não aparece no material): React Router, hooks (useState/useEffect), axios, Redux, TypeScript, banco SQL/Mongo, PHP (proibido). A troca de telas é feita com renderização condicional a partir do state.

Decisão sobre métodos HTTP: os exemplos usam só GET e POST. Para ficar 100% dentro do material, edição e exclusão usam POST em rotas próprias (ex.: `/turmas/excluir`).

- [ ] Perguntar ao professor no fórum se React Router e PUT/DELETE são aceitos (opcional; o plano funciona sem eles)

3. Escopo

Dois perfis (Coordenação e Aluno), cinco entidades e quatro casos de uso obrigatórios; os casos extras só entram se sobrar tempo.

Entidades

Entidade	Atributos
Aluno	id, nome, matricula, cursadas (ids de disciplinas)
Professor	id, nome, email, cargaMaxima (nº de turmas)
Disciplina	id, codigo, nome, creditos, preRequisitos (ids)
Turma	id, disciplinaId, professorId, horario (ex.: "21"), vagas, semestre
Matricula	id, alunoId, turmaId, data

Código de horário: 1º dígito = dia (2 seg, 3 ter, 4 qua, 5 qui, 6 sex, 7 sáb); 2º = turno (1 manhã, 2 tarde, 3 noite). O enunciado só define 1 e 3; "2 = tarde" é suposição nossa e deve ser explicada no vídeo.

Requisitos funcionais

1. RF01 — Entrar escolhendo perfil (Coordenação ou um aluno da lista)
2. RF02 — Listar, cadastrar, editar e excluir disciplinas com pré-requisitos
3. RF03 — Listar e cadastrar professores
4. RF04 — Ofertar turma (disciplina, professor, horário, vagas)
5. RF05 — Exibir grade de ofertas (dias × turnos)
6. RF06 — Aluno se matricular em turma
7. RF07 — Aluno cancelar matrícula
8. RF08 — Aluno ver "Minha grade" semanal

9. RF09 — Coordenação ver alunos de uma turma
10. RF10 — Coordenação ver carga de cada professor

Regras de negócio (validadas no front antes do fetch e de novo no Express)

Regra	Descrição	Caso de uso
RN01	Professor não tem duas turmas no mesmo horário	UC02
RN02	Professor não passa da cargaMaxima; o form sugere o de menor carga	UC02
RN03	Aluno só se matricula se cursou todos os pré-requisitos	UC03
RN04	Aluno não tem duas turmas no mesmo horário	UC03
RN05	Não há matrícula em turma lotada	UC03
RN06	Aluno não se matricula duas vezes na mesma disciplina	UC03
RN07	Horário válido: dia 2–7 e turno 1–3	UC02

Casos de uso

UC	Ator	Fluxo principal	Regras	Prioridade
UC01 Manter disciplinas	Coordenação	Lista → formulário → salvar → lista atualizada	—	Obrigatório
UC02 Ofertar turma	Coordenação	Escolhe disciplina, professor, dia, turno, vagas → valida → salva	RN01, RN02, RN07	Obrigatório
UC03 Realizar matrícula	Aluno	Vê grade de ofertas → clica Matricular → valida → confirma	RN03–RN06	Obrigatório
UC04 Minha grade / cancelar	Aluno	Vê grade semanal → cancela uma turma	—	Obrigatório
UC05 Manter professores	Coordenação	Lista + cadastro	—	Extra
UC06 Painel de carga e alunos por turma	Coordenação	Tabela de carga; detalhe da turma	—	Extra

Fluxos alternativos do UC03: se uma regra falhar, a matrícula não é feita e um alerta Bootstrap mostra o motivo ("Falta pré-requisito: Algoritmos I", "Conflito com a turma 33", "Turma lotada").

4. Arquitetura e pastas

Mesma estrutura do exemplo React05 da Semana 6: app create-react-app na raiz e uma pasta `backend/` com Express. O front roda na porta 3000 e chama o back na porta 3001 via fetch; o back lê e grava `dados.json`.

```
matriculas/
├── backend/
|   ├── package.json        (express, cors)
|   ├── server.js           (rotas + regras de negócio)
|   └── dados.json          (alunos, professores, disciplinas, turmas,
matriculas)
├── public/
|   └── index.html          (link do CSS do Bootstrap)
├── src/
|   ├── index.js
|   ├── App.js              (Provider do Context + ErrorBoundary + troca de
telas)
|   ├── AppContext.js       (createContext)
|   ├── api.js              (funções com fetch)
|   ├── regras.js           (validações RN01–RN07 no front)
|   ├── componentes/
|   |   ├── ErrorBoundary.js
|   |   ├── Menu.js
|   |   ├── Card.js         (composição com props.children)
|   |   └── GradeHorarios.js (tabela dias × turnos, reutilizada)
|   └── telas/
|       ├── Login.js
|       ├── Disciplinas.js
|       ├── Professores.js
|       ├── OfertarTurma.js
|       ├── DetalheTurma.js
|       ├── OfertasAluno.js
|       └── MinhaGrade.js
└── package.json            (react-scripts)
```

Troca de telas sem router: o Context guarda `usuario` e `tela`. O `App` faz um `switch` em `tela` e renderiza o componente certo; o `Menu` chama `irPara('disciplinas')`. É renderização condicional, conteúdo da Semana 4.

Bootstrap: link do CSS no `public/index.html` (CDN), usando as classes `container`, `table`, `form-control`, `btn`, `alert`, `navbar`. Não precisa instalar react-bootstrap.

5. API do back-end

Só GET e POST, como nos exemplos do professor. O servidor carrega `dados.json` com `fs.readFile`, aplica a regra e grava com `fs.writeFile`; em caso de erro devolve status 400 e `{ erro: "mensagem" }`.

Método	Rota	Corpo enviado	O que faz
GET	/dados	—	Devolve tudo (alunos, professores, disciplinas, turmas, matriculas)
POST	/disciplinas	{ codigo, nome, creditos, preRequisitos }	Cria ou edita (se vier id)
POST	/disciplinas/excluir	{ id }	Exclui se não houver turma usando
POST	/professores	{ nome, email, cargaMaxima }	Cria professor
POST	/turmas	{ disciplinaId, professorId, horario, vagas }	Valida RN01, RN02, RN07 e cria
POST	/turmas/excluir	{ id }	Exclui se não houver matriculados
POST	/matriculas	{ alunoId, turmaId }	Valida RN03–RN06 e cria
POST	/matriculas/cancelar	{ id }	Remove a matrícula

Um único `GET /dados` simplifica o front: cada tela chama no `componentDidMount` e filtra o que precisa.

Exemplo de `dados.json` inicial (dados de teste para o vídeo):

```
{
  "alunos": [
    { "id": 1, "nome": "Ana Souza", "matricula": "2026001", "cursadas": [1]
},
    { "id": 2, "nome": "Bruno Lima", "matricula": "2026002", "cursadas": [] }
  ],
  "professores": [
```

```
    { "id": 1, "nome": "Carla Dias", "email": "carla@uni.br", "cargaMaxima":
3 },
    { "id": 2, "nome": "Diego Ramos", "email": "diego@uni.br", "cargaMaxima":
2 }
  ],
  "disciplinas": [
    { "id": 1, "codigo": "ALG1", "nome": "Algoritmos I", "creditos": 4,
"preRequisitos": [] },
    { "id": 2, "codigo": "ALG2", "nome": "Algoritmos II", "creditos": 4,
"preRequisitos": [1] },
    { "id": 3, "codigo": "WEB", "nome": "Programação Front-end", "creditos":
4, "preRequisitos": [] }
  ],
  "turmas": [
    { "id": 1, "disciplinaId": 2, "professorId": 1, "horario": "21", "vagas":
2 },
    { "id": 2, "disciplinaId": 3, "professorId": 2, "horario": "21", "vagas":
30 }
  ],
  "matriculas": []
}
```

Com esses dados dá para mostrar no vídeo: Bruno bloqueado por pré-requisito em ALG2, Ana bloqueada por conflito no horário 21 e turma lotada após 2 matrículas.

6. Telas e componentes

Sete telas, todas componentes de classe; cada uma carrega os dados no `componentDidMount` e guarda no `state`. O protótipo (rabiscado em papel, Figma ou print das telas prontas) entra no ZIP como planejamento.

Tela	Perfil	O que mostra	Conteúdos da AA demonstrados
Login	Todos	Select com "Coordenação" + lista de alunos, botão Entrar	Formulário controlado, Context
Disciplinas	Coordenação	Tabela + formulário com select múltiplo de pré-requisitos, editar/excluir	.map + key, onSubmit, setState
Professores	Coordenação	Tabela com carga atual / máxima + formulário	Lista, cálculo derivado do state

Tela	Perfil	O que mostra	Conteúdos da AA demonstrados
Ofertar turma	Coordenação	Form (disciplina, professor, dia, turno, vagas) + GradeHorarios com as turmas	Validação RN01/RN02/RN07, alerta condicional
Detalhe da turma	Coordenação	Dados da turma e lista de alunos matriculados	props, renderização condicional (lista vazia)
Ofertas (aluno)	Aluno	GradeHorarios com botão Matricular em cada turma, vagas restantes	onClick, validação RN03–RN06, fetch POST
Minha grade	Aluno	GradeHorarios só com as turmas do aluno, botão Cancelar	Reuso de componente, fetch POST

Componentes reutilizáveis

- `GradeHorarios` — recebe `turmas` por props e desenha uma tabela Bootstrap com turnos nas linhas (Manhã, Tarde, Noite) e dias nas colunas (Seg–Sáb). Recebe também `renderAcao` para mostrar o botão certo (Matricular ou Cancelar). Usado em três telas: é o melhor trecho para mostrar no vídeo.
- `Card` — moldura com título e `props.children` (composição).
- `Menu` — navbar que muda conforme o perfil do Context.
- `ErrorBoundary` — `getDerivedStateFromError` + `componentDidCatch`, mostra "Algo deu errado" sem travar a página.

Exemplo de esqueleto de tela (padrão a repetir):

```
class MinhaGrade extends Component {
  static contextType = AppContext;
  state = { dados: null, mensagem: '' };

  componentDidMount() {
    buscarDados().then(dados => this.setState({ dados }));
  }

  cancelar(matriculaId) {
    cancelarMatricula(matriculaId)
      .then(() => buscarDados())
      .then(dados => this.setState({ dados, mensagem: 'Matrícula cancelada'
}));
  }

  render() {
    if (!this.state.dados) return <p>Carregando...</p>;
    // filtra as turmas do aluno logado (this.context.usuario) e usa
<GradeHorarios />
  }
}
```

7. Cronograma (07/11 a 15/11)

Nove dias: back-end pronto até domingo, um caso de uso por dia até quinta, e sexta e sábado reservados para testes, vídeo e envio. Entregar na sexta deixa o sábado de folga para imprevistos e para usar a 2ª tentativa se precisar.

Sáb 07/11 — Planejamento e protótipo

- [ ] Revisar este plano e decidir: individual ou grupo (até 5)
- [ ] Desenhar as 7 telas (papel ou Figma) e salvar as imagens
- [ ] Criar o projeto: `npx create-react-app matriculas` e a pasta `backend/`

Dom 08/11 — Back-end

- [ ] `backend/`: `npm init -y`, `npm install express cors`
- [ ] `server.js` com `GET /dados` lendo `dados.json` (base: exemplo da Semana 5)
- [ ] Rotas POST de disciplinas, professores, turmas e matrículas com as regras RN01–RN07
- [ ] Testar cada rota pelo navegador ou por um `fetch` no console

Seg 09/11 — Base do front + UC01

- [ ] Bootstrap no `index.html`, `AppContext`, `ErrorBoundary`, `Menu`, `api.js`

- [ ] Tela Login e troca de telas pelo Context
- [ ] Tela Disciplinas completa (listar, criar, editar, excluir, pré-requisitos)

Ter 10/11 — UC02 Ofertar turma

- [ ] Componente `GradeHorarios`
- [ ] Tela Professores (UC05, simples)
- [ ] Tela Ofertar turma com validações RN01, RN02, RN07 e sugestão do professor de menor carga

Qua 11/11 — UC03 Realizar matrícula

- [ ] Tela Ofertas do aluno usando `GradeHorarios`
- [ ] Validações RN03–RN06 em `regras.js` + alertas Bootstrap
- [ ] Testar os 4 bloqueios com os dados de teste

Qui 12/11 — UC04 + extras

- [ ] Tela Minha grade com cancelamento
- [ ] Detalhe da turma e carga dos professores (UC06), se der tempo
- [ ] Revisar nomes, indentação e comentários curtos nas partes importantes

Sex 13/11 — Testes e vídeo

- [ ] Restaurar `dados.json` inicial e testar todos os fluxos do zero
- [ ] Gravar o vídeo seguindo o roteiro (seção 8)
- [ ] Montar o ZIP e enviar a tentativa 1

Sáb 14/11 e Dom 15/11 — Folga

- [ ] Conferir se o envio abriu certo; corrigir e reenviar só se necessário (prazo 15/11, 23:59)

8. Vídeo e entrega

O vídeo tem no máximo 5 min: cerca de 2 min de sistema funcionando e 2,5 min de código, com zoom no editor (fonte 18 ou mais) para ficar legível.

Tempo	O que mostrar	O que falar
0:00–0:20	Tela inicial	Nome(s), tema, tecnologias (React, Bootstrap, Express)
0:20–1:10	Coordenação: cadastra disciplina com pré-requisito, oferta turma, tenta conflito de professor	UC01, UC02, RN01/RN02

Tempo	O que mostrar	O que falar
1:10–2:10	Aluno: matrícula ok, bloqueios (pré-requisito, conflito, lotada), cancela em Minha grade	UC03, UC04, RN03–RN06
2:10–2:40	`server.js` + `dados.json`	Express, rotas GET/POST, fs grava o JSON
2:40–3:20	`api.js` e uma tela com `componentDidMount` + `setState`	fetch, Promise, ciclo de vida
3:20–4:00	`regras.js` e `GradeHorarios`	Regras de negócio, `.map` + key, props, reuso
4:00–4:30	`App.js`, Context, ErrorBoundary	Troca de telas por renderização condicional, contexto global
4:30–4:50	Encerramento	Resumo dos casos de uso entregues

Gravação: OBS Studio ou o gravador do Windows (Win+Alt+R) já salvam em MP4. Confira se ficou entre 3 e 5 min.

Montar o ZIP

- [ ] Apagar `node_modules/` da raiz e de `backend/` (o professor roda `npm install`)
- [ ] Incluir um `LEIAME.txt`: como rodar (`cd backend && npm install && node server.js`; na raiz `npm install && npm start`)
- [ ] Incluir pasta `planejamento/` com o PDF deste plano e as imagens das telas
- [ ] Compactar como `.zip`, não .rar nem .7z

Envio (só 1 arquivo, até 150 MB)

- [ ] Se o vídeo couber, colocar o MP4 dentro do ZIP junto com o código
- [ ] Se não couber, perguntar ao professor como enviar o vídeo (o enunciado não diz; link de repositório não é aceito para o código)
- [ ] Escrever o nome de todos os integrantes no campo de resposta
- [ ] Enviar a tentativa 1 até 13/11